Documento de Diseño Técnico - Caso: Predicción de Abandono de Clientes (Telco Customer Churn)

Contenido

Introducción 3
1. Descripción técnica del sistema 3
    1.1 Componentes técnicos del sistema 3
    1.2 Estructura general 3
    1.3 Tecnologías utilizadas 4
    1.4 Relación entre componentes 4
2. Diagrama técnico de la solución 5
3. Modelo de datos básico y diccionario 6
    3.1 Modelo de datos 6
    3.2 Diccionario de datos 6
4. README.md actualizado y versionado 7
Cierre 7

Introducción

Este documento describe de forma técnica y estructurada la arquitectura del sistema desarrollado para la predicción de abandono de clientes (*Churn*). Contiene los elementos necesarios para comprender la implementación del pipeline de datos, desde la ingesta de registros hasta el despliegue del modelo predictivo, sirviendo como guía para la replicabilidad y mantenimiento de la solución.

1. Descripción técnica del sistema

1.1 Componentes técnicos del sistema

Para cumplir adecuadamente con el ciclo de vida de los datos, los componentes se agrupan en las siguientes etapas clave:

- **Etapa de Ingesta:** Módulo de scripts en Python encargados de la lectura de datos crudos (CSV) y tratamiento de anomalías iniciales.
- **Etapa de Procesamiento:** Módulo de transformación para el preprocesamiento de variables categóricas y escalamiento numérico, en conjunto con el módulo de Inteligencia Artificial encargado del entrenamiento y evaluación del modelo de clasificación.
- **Etapa de Servicio:** Base de datos NoSQL (MongoDB) para almacenar de forma persistente los perfiles procesados, y una infraestructura basada en contenedores Docker con hosting en Render para disponibilizar y servir el modelo predictivo a los usuarios finales.

1.2 Estructura general

La solución adopta una arquitectura modular basada en canalizaciones (pipelines) de datos. Se descarta un diseño monolítico en favor de scripts especializados. Cada etapa del flujo cumple una función única, procesa la información y la transfiere al siguiente eslabón.

Esta elección de arquitectura se justifica técnicamente porque garantiza el cumplimiento de los siguientes atributos de calidad:

- **Escalabilidad:** Al separar la ingesta, el procesamiento y el servicio, es posible aumentar los recursos de cómputo en Render o la capacidad de almacenamiento en MongoDB de forma independiente según la demanda, sin generar cuellos de botella en el resto del sistema.

- **Seguridad:** El manejo de la información y los accesos se encuentra protegido mediante entornos aislados. Las credenciales de conexión a MongoDB y los tokens de despliegue no se exponen nunca en el código fuente, sino que se gestionan y proveen de forma segura mediante los *secrets* de GitHub Actions durante la validación CI/CD.
- **Interoperabilidad:** Al estructurar la información mediante documentos desnormalizados tipo JSON y encapsular todo el ecosistema en contenedores Docker, se asegura que el modelo predictivo pueda interactuar fluidamente con otras aplicaciones y ejecutarse de manera idéntica en cualquier plataforma de despliegue.

1.3 Tecnologías utilizadas

Python 3: Lenguaje principal para el procesamiento lógico y el modelado analítico.

MongoDB: Motor de base de datos NoSQL. Se seleccionó por su capacidad para manejar documentos JSON, otorgando la flexibilidad necesaria para adaptar los atributos de los clientes sin la rigidez de las tablas relacionales tradicionales.

Docker: Herramienta de virtualización que estandariza el entorno de ejecución, garantizando que el código opere de manera idéntica en cualquier estación de trabajo local.

GitHub Actions: Orquestador de integración continua que evalúa el código de forma automatizada antes de su despliegue.

Render: Proveedor de servicios en la nube seleccionado para alojar el *backend* predictivo de manera ágil.

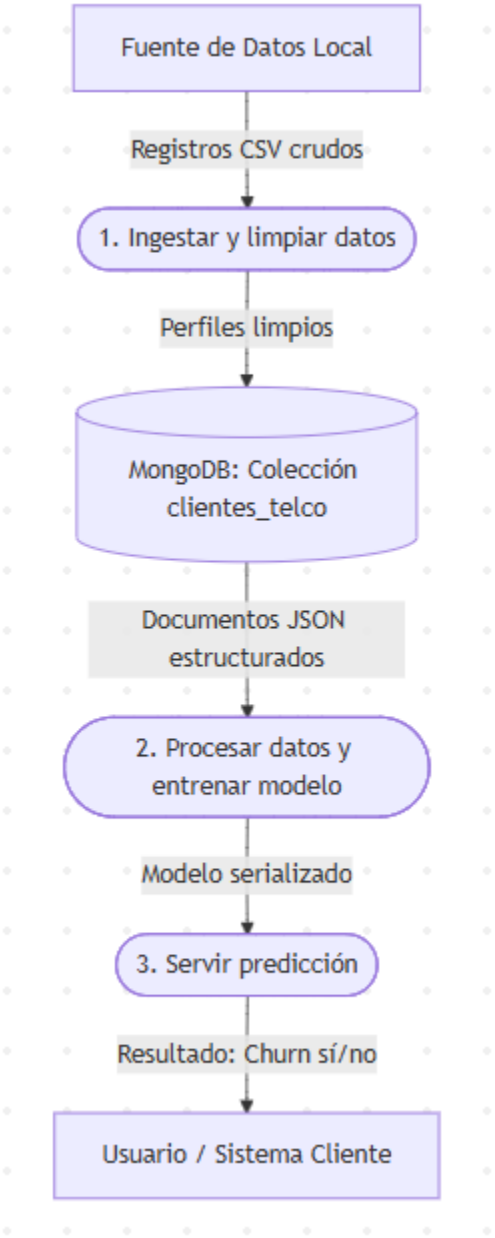
VS Code y Codespaces: Entorno de desarrollo centralizado para estandarizar el espacio de trabajo de los ingenieros involucrados.

1.4 Relación entre componentes

El proceso inicia cuando el script de Python lee el archivo CSV local y ejecuta las reglas de limpieza. Inmediatamente después, el mismo módulo establece una conexión con MongoDB para insertar los registros procesados. Con la base de datos operativa, el módulo de *Machine Learning* extrae esa información estructurada para el entrenamiento del modelo. Todo este ecosistema de Python y MongoDB se ejecuta dentro de un contenedor Docker. Cualquier modificación a estos scripts se envía a GitHub, donde GitHub Actions valida su correcta ejecución antes de empaquetar la solución y enviarla al servidor de producción en Render.

2. Diagrama técnico de la solución

El siguiente diagrama representa el recorrido de la información a través del sistema:



El flujo inicia en el entorno local con la ingesta y limpieza de los registros crudos (CSV). Una vez procesados, se almacenan como documentos JSON estructurados en la colección `clientes_telco` de MongoDB. Posteriormente, el módulo de Machine Learning extrae estos datos para el entrenamiento del modelo. Finalmente, el sistema entrega la predicción de abandono (Churn) al usuario final, operando sobre una infraestructura segura y escalable orquestada con Docker.

3. Modelo de datos básico y diccionario

3.1 Modelo de datos

Al implementar MongoDB, el modelo de datos prescinde de diagramas Entidad-Relación y tablas fragmentadas. La arquitectura se centra en una única colección denominada `clientes_telco`. Dentro de esta colección, la información se almacena mediante documentos estructurados (tipo JSON), donde cada registro consolida el identificador del cliente, sus características demográficas, sus servicios contratados y la variable objetivo (*Churn*). Esta estructura desnormalizada agiliza la extracción y el consumo de datos durante la fase de entrenamiento del algoritmo.

3.2 Diccionario de datos

Campo	Tipo de dato	Descripción	Valores esperados / restricciones	¿Obligatorio?
customerID	STRING	Identificador único del cliente (Clave principal).	Alfanumérico Único por documento.	Sí
tenure	INT	Número de meses que el cliente ha permanecido en la compañía.	Entero >= 0.	Sí
InternetService	STRING	Tecnología base del servicio de internet contratado.	dsl, fibra_optica, no	Sí
OnlineSecurity	BOOLEAN	Indica si el cliente tiene suscripción de seguridad en línea.	1=si, 0=no	Sí
TechSupport	BOOLEAN	Indica si el cliente cuenta con soporte técnico prioritario.	1=si, 0=no	Sí
Contract	STRING	Tipo de contrato o modalidad de permanencia.	mes_a_mes, un_anio, dos_anios	Sí
PaymentMethod	STRING	Método transaccional para el pago mensual.	cheque_elec, cheque_correo, trans_bancaria, tarjeta_credito	Sí
MonthlyCharges	INT	Monto facturado mensualmente al	Entero > 0.	Sí

		cliente.		
TotalCharges	INT	Monto total acumulado facturado durante su permanencia.	Entero > 0.	Sí
Churn	STRING	**Variable objetivo: Indica si el cliente abandonó el servicio.**	si, no	Sí

4. README.md actualizado y versionado

Como respaldo técnico fundamental, se ha estructurado un archivo `README.md` en la raíz del repositorio oficial del proyecto. Este documento detalla la arquitectura implementada, lista las herramientas tecnológicas clave y proporciona las instrucciones precisas de consola para compilar el contenedor Docker y ejecutar la canalización de datos en cualquier estación de trabajo local.

Cierre

Este documento representa el diseño técnico de base para el desarrollo del sistema de predicción de abandono de clientes (*Churn*). Establece la arquitectura del Pipeline Híbrido, justifica las decisiones tecnológicas adoptadas y define el esquema exacto de los datos a procesar.

Debe ser considerado un documento vivo; por lo tanto, será actualizado si durante las fases de desarrollo e implementación se incorporan nuevas herramientas, se modifican los modelos matemáticos o se aplican cambios estructurales en la base de datos MongoDB. Al finalizar esta fase analítica, tanto este documento como el archivo `README.md` y el código fuente inicial quedarán subidos, sincronizados y versionados permanentemente en el repositorio oficial de GitHub del equipo, asegurando así la replicabilidad técnica y la transparencia del proyecto.